Embarcatech

Hardware BR

# Projeto de Encerramento da Fase III

Guilherme Salles Blanco

Brasília, 2026

# Resumo

Este trabalho apresenta o processo completo de portabilidade, síntese e validação do processador RISC-V *TinyQV*, um núcleo educacional e colaborativo desenvolvido no ecossistema *TinyTapeout*, para execução em uma FPGA real. Ele possui como objetivo o funcionamento completo de um programa originalmente escrito na linguagem C dentro do processador carregado na FPGA *Colorlight-i9*, seguindo um pipeline de síntese e desenvolvimento completamente *open source*.

A revisão bibliográfica trata da arquitetura aberta RISC-V, do ecossistema *TinyTapeout* e de outras ferramentas, como o *Yosys* e o *OpenFPGALoader*, além do SDK do *TinyQV* e do *toolchain* RISC-V.

A metodologia envolve etapas incrementais, como: adaptação de módulos *Verilog*, instalação e correção do *toolchain*, migração do fluxo de síntese para plataformas diferentes, compilação de programas C para o formato .hex, entre outras etapas. A validação final consistiu na execução de um programa C que controla um LED conectado ao pino J16 da placa.

Os resultados confirmam a síntese correta do núcleo, a geração de diagramas RTL e físicos coerentes e a execução correta de programas C, comprovando o funcionamento do processador e validando o sistema. Ele demonstra a viabilidade prática de utilizar arquiteturas RISC-V em FPGAs, ainda que periféricos mais complexos não tenham sido testados.


# 1) Introdução

## 1.a) Contextualização

Nos últimos anos, o progresso do mundo tecnológico foi tendenciando-se à migração e utilização de arquiteturas abertas (*open source*), que, quando utilizadas corretamente, podem resultar na redução de custos de licenciamento para os desenvolvedores, independência tecnológica do projeto realizado e a possibilidade de modificação profunda delas.

Um exemplo de arquitetura aberta, modular e livre de *royalties* é o RISC-V (*Reduced Instruction Set Computer*, ou Computador com Conjunto Reduzido de Instruções), que tem se consolidado como uma alternativa estratégica para pesquisa, desenvolvimento, indústria e ensino, permitindo a implementação e elaboração de processadores para dispositivos microcontrolados e sistemas embarcados sem as barreiras encontradas em arquiteturas proprietárias.

Nos últimos anos, o RISC-V passou de uma mera iniciativa acadêmica a elemento central da indústria global de semicondutores. O mercado mundial de tecnologias baseadas em RISC-V movimentou US$ 1,76 bilhão em 2024, com projeção de atingir US$ 25,73 bilhões até 2034, e o Brasil está dentro da lista de países emergentes neste mercado, juntamente com a índia, Argentina e África do Sul. [11]

Adicionalmente, um outro ponto importante para o avanço da tecnologia é a democratização de ferramentas de prototipagem e fabricação de circuitos integrados. Um exemplo de iniciativa que visa a combater o custo e a complexidade associados à fabricação de ASICs (*Application-Specific Integrated Circuit*, ou Circuito Integrado de Aplicação Específica) é a *TinyTapeout*, que regularmente realiza projetos que incentivam a submissão de circuitos programados em linguagens de descrição de *hardware*, como o VHDL e o Verilog, feitos por estudantes e pesquisadores, que são então transformados em chips compostos por diversos ASICs. Isso

promove a inclusão tecnológica e experimentação prática para qualquer tipo de pessoa, sem a necessidade de realizar o envio tradicional de chips, o que pode custar centenas de milhares de dólares. Porém, para que um design seja submetido ao fluxo de fabricação de circuitos integrados, é necessário validá-lo em *hardware* reconfigurável, como FPGAs.

Um dos projetos desenvolvidos dentro do ecossistema do *TinyTapeout* é o *TinyQV*, um processador RISC-V *Crowdsourced* projetado para fins educacionais, que contou e continua contando com a participação de vários desenvolvedores que criam periféricos que se comunicam com o processador em si.

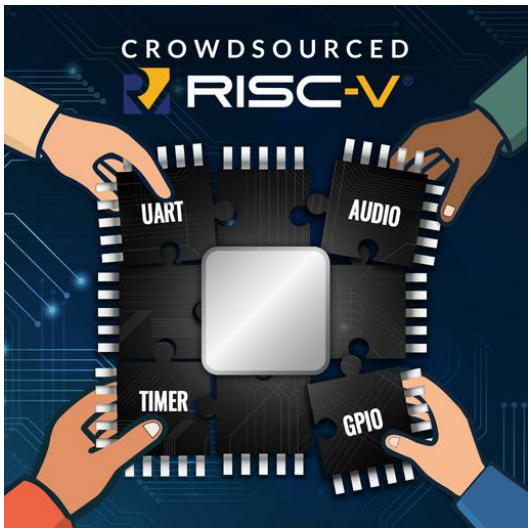


**Figura 1:** Imagem ilustrativa do processador RISC-V *TinyQV.* Fonte: [1]

Apesar da expansão internacional dessas iniciativas, ainda há baixa participação brasileira em projetos de *hardware open source* e no ecossistema de fabricação acessível. Uma das dificuldades encontradas é a falta de experiência prática com portabilidade de circuitos, como processadores RISC-V, e a validação de sistemas embarcados em plataformas reconfiguráveis. Assim, torna-se importante desenvolver experiência prática em portabilidade de processadores *open source* e validação de *hardware* digital para possibilitar a participação efetiva em iniciativas de fabricação acessível como o *TinyTapeout*.

## 1.b) Definição do problema

Apesar da existência do *TinyQV*, sua utilização fora do ambiente de simulação e testes é limitada. Para que esse processador possa ser empregado de forma mais ampla, é necessário torná-lo executável em uma plataforma física, como uma FPGA comercial.

O processo de portar um processador real para uma FPGA envolve alguns desafios técnicos, como a adaptação do código *Verilog*/VHDL ao sintetizador-alvo, correção de incompatibilidades, integração com memórias presentes na placa, criação de um fluxo de geração de *bitstream* estável, teste do pipeline com *software* embarcado interagindo com algum dispositivo físico, entre outras etapas.

O problema tratado neste projeto consiste na necessidade de portar, validar e testar o processador *TinyQV* em uma FPGA real que, neste caso, é a Lattice ECP5-45F presente na placa de desenvolvimento Colorlight-i9, que está representada visualmente na figura 2 e esquematicamente, por meio de um *pinout*, na figura 3.

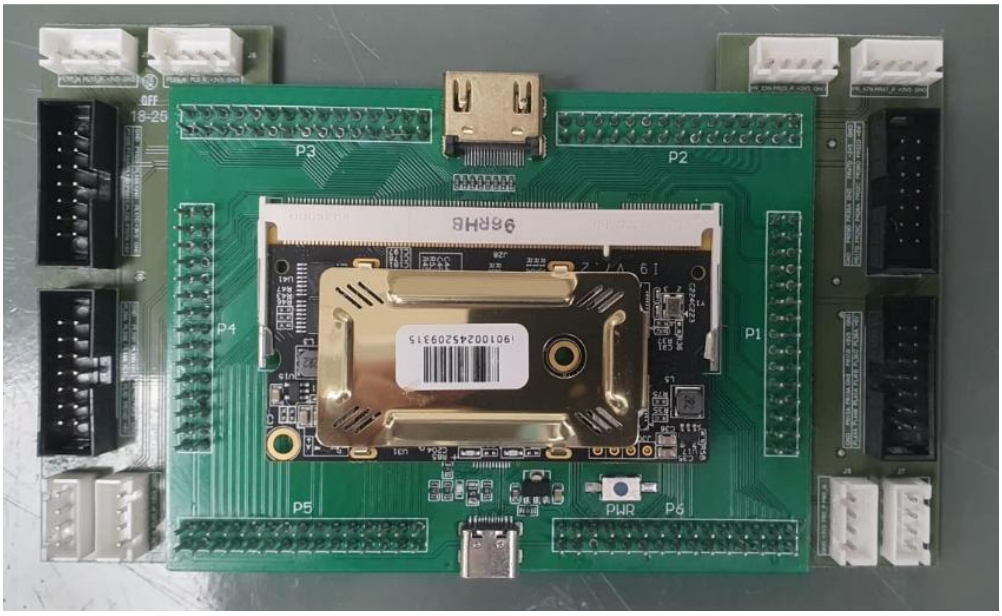


**Figura 2:** Placa Colorlight i9. Fonte: [3].

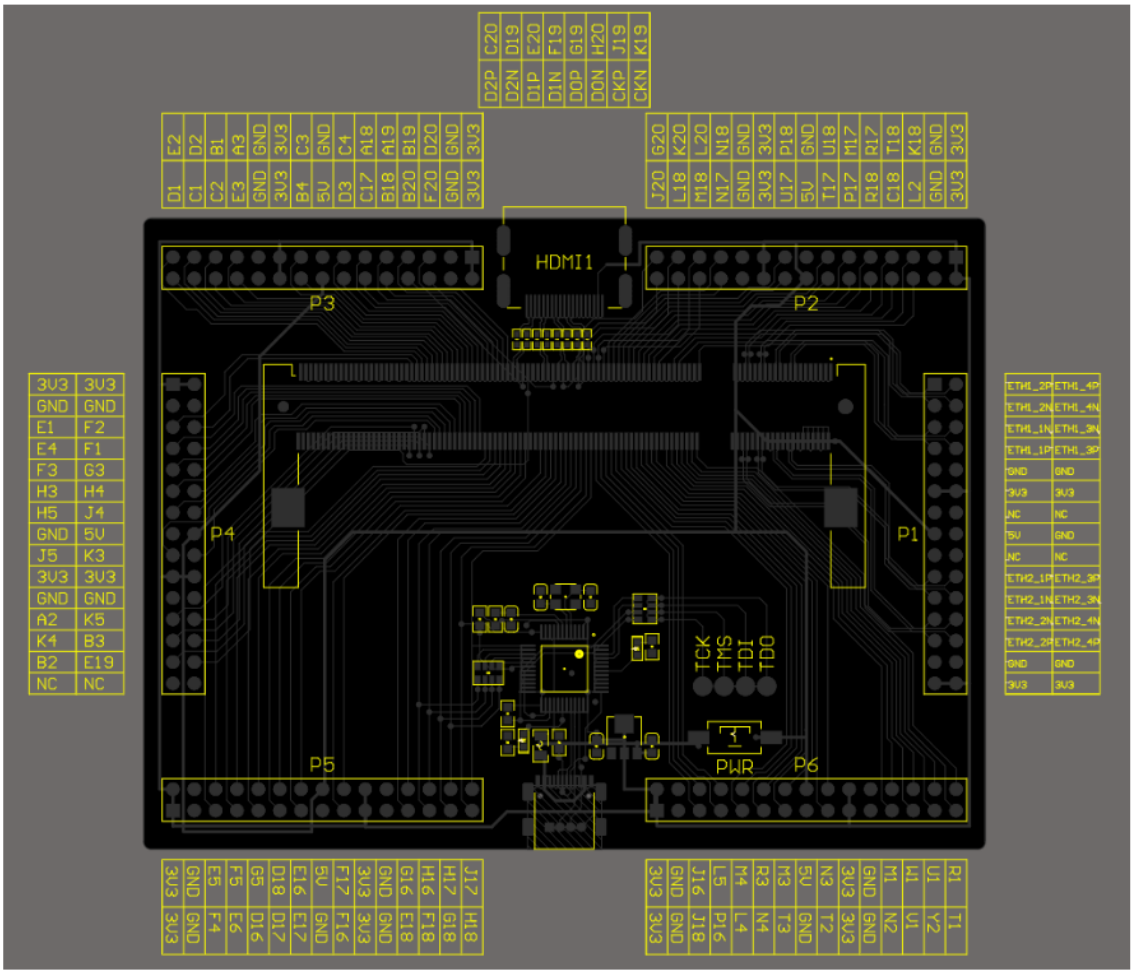


**Figura 3:** Pinout da placa Colorlight-i9. Fonte: [3].

O processador deve, ao final do projeto, ser capaz de executar programas escritos na linguagem de programação C, permitindo testes práticos e servindo como base para criações futuras, como a criação de novos periféricos ou a submissão de projetos ao *TinyTapeout*. Resolver esse

problema ajuda a ampliar a adoção de arquiteturas abertas no Brasil, fortalecer a formação em sistemas embarcados avançados e permitir que novos projetos possam evoluir.

## 1.c) Objetivos do trabalho

O objetivo geral deste trabalho foi realizar o processo de implementação e validação do *TinyQV* em uma FPGA. Não é a criação de uma arquitetura RISC-V modificada, ou a elaboração de um processador completo, mas sim uma forma de comprovar a portabilidade do *TinyQV* e que processadores RISC-V são viáveis para uso prático em uma plataforma FPGA real, podendo ser executados de forma independente com programas escritos em C.

Os objetivos específicos são:

1. Adaptar os arquivos Verilog do *TinyQV* para o fluxo de síntese compatível com a FPGA Lattice ECP5-45.
2. Integrar módulos essenciais ao funcionamento do processador, incluindo uma memória virtual RAM/ROM baseada no módulo sim_qspi.v. [1]
3. Configurar e instalar o *toolchain* RISC-V e do *Source Development Kit* (SDK) do *TinyQV*, que permitem a compilação de códigos C e C++ em arquivos .hex, .elf e .bin para que sejam executados pelo processador *TinyQV*. [4] [5]
4. Elaborar e ajustar o fluxo de geração de *bitstream* com definição de *constraints* e portas em um arquivo LPF.
5. Carregar e executar o *bitstream* na Colorlight-i9, iniciando o processo de validação funcional.
6. Criar um programa em C para ser executado no *TinyQV*, compilado com o SDK e o *toolchain*. O programa deve, necessariamente, realizar alguma interação com o mundo real.

## 1.d) Proposta de solução

Para o funcionamento da solução, o *TinyQV* é tratado como um bloco funcional dentro da FPGA, com comunicação com um bloco de memória definido pelo módulo *Verilog* sim_qspi.v. Para que o núcleo RISC-V funcione corretamente, ele utiliza os sinais de *Clock* e *Reset* da própria FPGA (pinos P3 e J18, respectivamente).

Adicionalmente, é necessária uma fonte de alimentação para a FPGA, como a conexão USB entre a placa e um computador, e os periféricos de saída e de entrada, que interagem com o programa executado dentro do processador. Um exemplo de periférico de saída, que será utilizado como dispositivo de validação, é um LED externo à placa.

Na figura 3 apresenta-se o diagrama esquemático que resume a solução proposta.

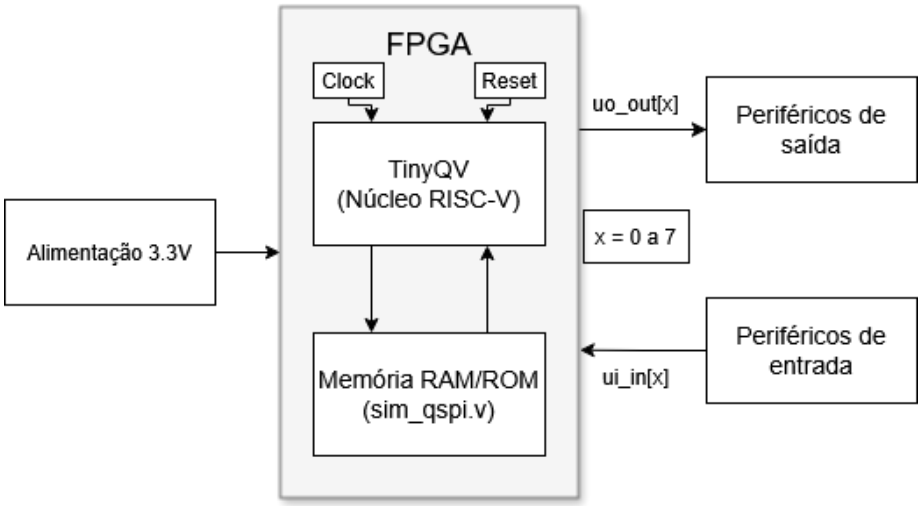


**Figura 4:** Diagrama de funcionamento do projeto. Fonte: Própria.

## 1.e) Detalhamento do documento

Os seguintes capítulos apresentam o processo de desenvolvimento completo do projeto. No capítulo 2, é realizada a revisão bibliográfica dos temas relevantes ao trabalho, como a arquitetura RISC-V, o núcleo *TinyQV* e as ferramentas utilizadas. O capítulo 3 trata da metodologia incremental adotada que foi utilizada ao longo do projeto. O capítulo 4 reúne os resultados obtidos, como os diagramas de síntese física e RTL e a validação física do sistema na FPGA. Por fim, o capítulo 5 apresenta as conclusões, limitações encontradas e sugestões de trabalhos futuros.

# 2) Revisão bibliográfica

O desenvolvimento deste trabalho envolveu tecnologias e ferramentas relacionadas à implementação de processadores em plataformas reconfiguráveis. Uma dessas tecnologias que precisou ser estudada e compreendida para sua utilização no projeto é a arquitetura aberta RISC-V, que, por possuir uma arquitetura de conjunto de instruções (*Instruction Set Architecture*, ou ISA) livre de *royalties*, permite que pesquisadores explorem diferentes tipos de implementações sem enfrentar restrições de licenciamento. Processadores e produtos criados com base no conjunto de instruções do RISC-V podem ser licenciados posteriormente, mas as instruções em si permanecerão abertas e livres para qualquer time de *chip designers*, e estão presentes no repositório do RISC-V [7], administrado pelo comitê de desenvolvimento *RISC-V International* da Suíça.

O *TinyQV* é um dos processadores criados pela iniciativa *TinyTapeout* a partir da ISA do RISC-V. Sua principal característica é o fato de seu *core* (núcleo) estar finalizado, enquanto módulos e periféricos que realizam diversos tipos de tarefas, como comunicação com dispositivos eletrônicos externos, são desenvolvidos colaborativamente por diversos contribuidores. [1]

O núcleo do *TinyQV* é minimalista e compatível com arquiteturas RV32EC e possui suporte a interrupções básicas. Adicionalmente, possui uma interface conjunta QSPI de memória *flash* e RAM, alguns periféricos básicos como UART, SPI e um único canal PWM, além de alguns GPIOs. O mapa de endereços, que são de 28 bits, é:

- 0x0000000 - 0x0FFFFFF: Flash

- 0x1000000 - 0x17FFFFF: RAM A
- 0x1800000 - 0x1FFFFFF: RAM B
- 0x8000000 - 0x80007FF: Registradores de periféricos

Para a síntese de módulos *Verilog* em arquivos *bitstream* que são colocados dentro da FPGA, foi considerado, inicialmente, a utilização do programa *Lattice Diamond*. Essa consideração foi feita pelo fato de ele possuir uma interface de usuário gráfica e ser naturalmente compatível com a FPGA utilizada no projeto. Porém, por conta de erros de compatibilidade do código disponibilizado pela *TinyTapeout* com a estrutura sintática demandada pelo *Lattice Diamond*, foi realizada a troca de sintetizador para o *Yosys* [8]. Para que o *Yosys* realizasse, também, o processo de posicionamento e roteamento específico para a família ECP5, foi necessário a ferramenta *nextpnr-ecp5* [9]. Para carregar o *bitstream* diretamente na placa *Colorlight-i9*, foi utilizado o programa *openFPGALoader* [10].

# 3) Metodologia

A metodologia deste projeto se baseou em um conjunto de experimentos incrementais organizados em etapas. A primeira etapa consistiu na importação dos códigos *Verilog* do núcleo do *TinyQV* e dos periféricos que já haviam sido criados por participantes da iniciativa. Inicialmente, foram baixados apenas os módulos necessários para o funcionamento do processador, mas percebeu-se que quase todos os módulos presentes nos repositórios do *TinyTapeout* seriam necessários, como o *sim_qspi.v*. Estes arquivos foram colocados no *Lattice Diamond*, tiveram sua sintaxe ajustada e o *bitstream* foi criado corretamente, embora ainda sem ser carregado ou testado na placa física, pois o arquivo LPF não havia sido criado ainda.

Havia um receio de utilizar o arquivo *peripherals.v*, pois entendeu-se que ele possuía ou chamava todos os periféricos criados pelos participantes da iniciativa, então foi criado um arquivo *peripherals_min.v*, que possuía apenas as chamadas dos periféricos essenciais, como UART e GPIO. Porém, ao contrário do que havia sido entendido anteriormente, *peripherals.v* possui diversos módulos vazios escritos especificamente para que os periféricos sejam inseridos de acordo com a necessidade do usuário, e como o *peripherals_min.v* não estava funcionando corretamente, o *peripherals.v* foi incluído no projeto sem qualquer edição adicional.

Dentro do repositório dos códigos do *TinyQV*, em uma seção específica para os códigos que precisavam ser carregados em FPGAs genéricas (que não eram do tipo *Pico-ICE*), estava um programa de exemplo que seria carregado dentro do processador, este sendo o *ledstrip.hex*. Embora ele não pudesse ser testado fisicamente, pois necessitava de um componente eletrônico específico, ele ainda foi colocado dentro do projeto para testar se o processador estava recebendo o arquivo .hex contendo o programa escrito em C. O sintetizador não apresentou erros quando o programa foi introduzido a ele, mas os testes com este programa em específico não prosseguiram depois desse ponto. O *Lattice Diamond* apresentou as seguintes imagens que representam o projeto sintetizado:

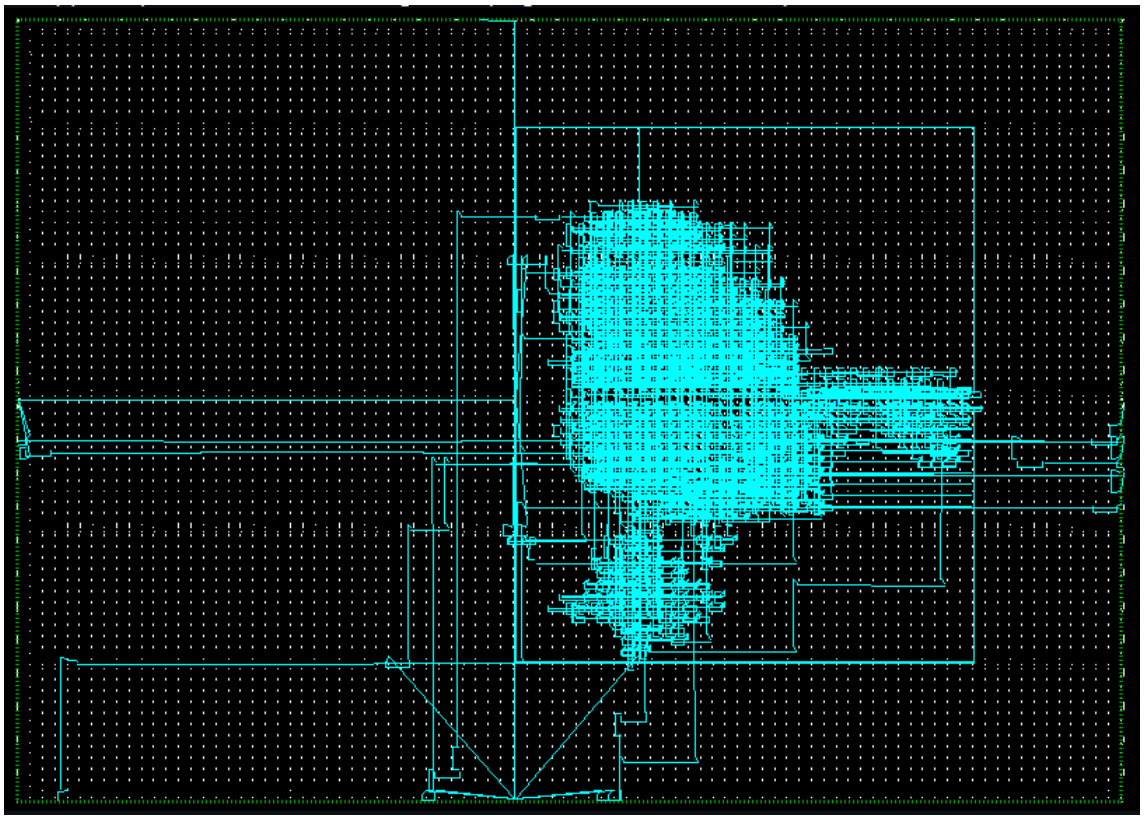

**Figura 5:** *Physical View* do Processador dentro do *Lattice Diamond*. Fonte: Própria.

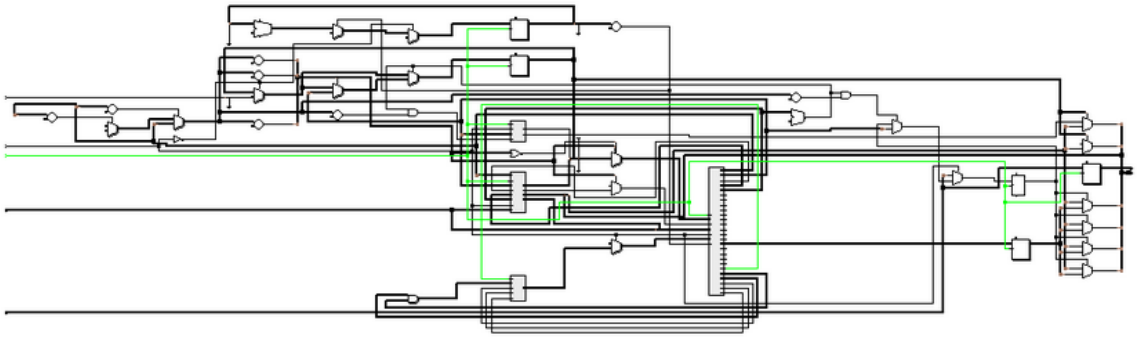

**Figura 6:** Diagrama do módulo *top.v* do processador dentro do *Lattice Diamond.* Fonte: Própria.

Para permitir testes funcionais reais, foi necessário instalar o *toolchain* RISC-V e o SDK específico do *TinyQV*. Foram realizadas três tentativas de instalação, cada uma envolvendo versões diferentes do *toolchain* e formas diferentes de integrar o SDK a ela. Na primeira instalação, a *toolchain* não funcionava ou aceitava programas escritos em C. Na segunda instalação, os programas eram carregados nela, que gerava os arquivos .hex, .elf e .o, contanto que o programa utilizasse funções básicas e nativas da linguagem C, sem utilizar qualquer tipo de biblioteca ou instrução externa. Isso ocorreu pois o *toolchain* havia sido compilado para RV32I com ABI ilp32, enquanto o *TinyQV* requer a arquitetura RV32EC com ABI ilp32e. Na terceira e última instalação, programas utilizando bibliotecas e instruções externas estavam sendo corretamente compilados e transformados no tipo de arquivo que o processador precisava.

Durante todas essas instalações, foram revisados certos arquivos *Makefile* que realizavam o processo de compilação e tradução dos programas em C para .hex, .elf e .o. A base desses arquivos é do próprio *Makefile* disponibilizado nos repositórios do SDK.

Após isso, foram realizados novos testes com programas escritos em C e carregados na placa. O primeiro teste foi fazer o LED disponível na própria *Colorlight-i9* acender, mas a sintetização dos códigos *Verilog* estava dando alguns novos problemas no *Lattice Diamond*. Assim, após mostrar-se inadequado para as exigências do *TinyQV*, o fluxo de síntese do *Lattice Diamond* foi trocado pelo fluxo do *Yosys*. Para isso, o *Makefile* original do *TinyQV* foi ajustado para geração de *bitstream* compatível com a placa, além da inclusão do arquivo LPF. Os *bitstreams* dos testes elaborados foram carregados na placa por meio do *openFPGALoader*.

A última etapa da metodologia consistiu no teste físico final do sistema implementado. A tarefa que a placa deveria realizar era, por meio do processador do *TinyQV*, executar um simples programa em C que faria um LED externo piscar em uma certa frequência determinada pelo programa. Isso permitiu avaliar e validar a execução correta de instruções, a operação da memória, a comunicação entre *software* e *hardware*, a estabilidade do *clock* e do *reset* e o acesso aos registradores do *TinyQV*.

A tabela a seguir resume a metodologia realizada:

| Etapa | O que foi testado | Critério de validação |
|---|---|---|
| Inclusão inicial dos arquivos | Síntese mínima do TinyQV | Bitstream gerado |
| Instalação do toolchain | Compilação de C para .hex/.elf | Arquivos .hex/.elf gerados |
| Ajustes do SDK | Testes com programas oficiais | Programas compilam |
| Migração para Yosys | Fluxo open-source funcional | Bitstream gerado e carregado na placa |
| Teste em *hardware* | Execução real do LED | LED piscando via programa C |

**Tabela 1:** Resumo da metodologia realizada. Fonte: Própria.

# 4) Resultados

A figura 7 apresenta uma parte do diagrama esquemático do projeto, gerado a partir da síntese RTL no Yosys. O diagrama completo possui dimensões muito superiores ao permitido para visualização clara em um documento e está disponibilizado integralmente no link a seguir: https://github.com/GuilhermeSallesBlanco/ProjetoTinyQV/blob/main/tinyqv_rtl.svg.

A parte demonstrada na figura 7 apresenta a interconexão entre os blocos centrais de processamento e seleção de periféricos.

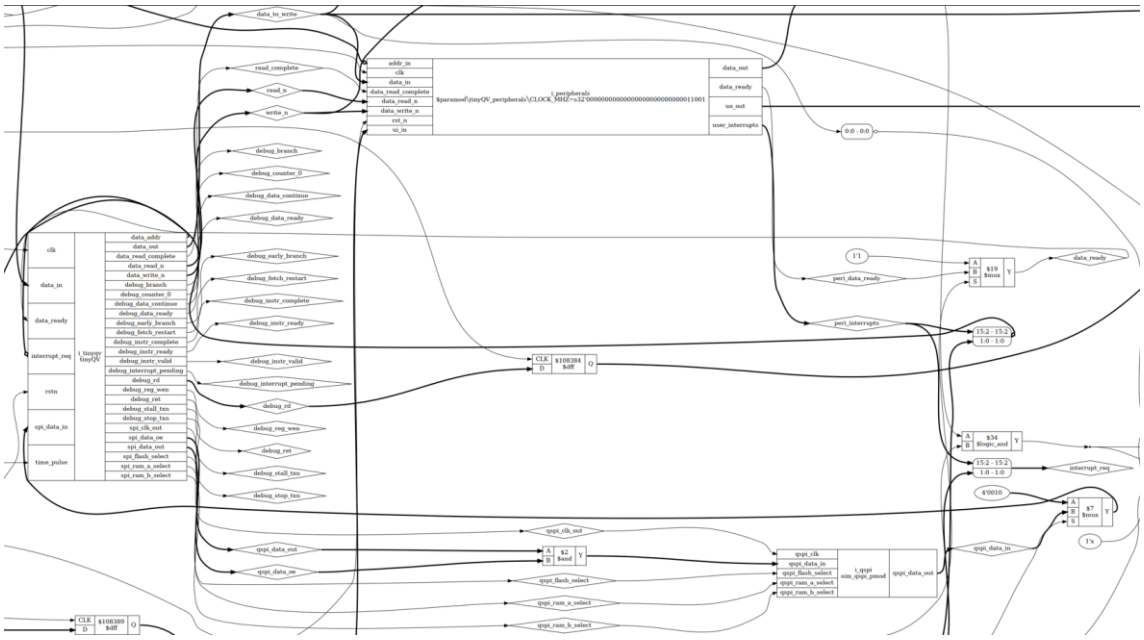

**Figura 7:** Diagrama esquemático gerado pela síntese RTL no Yosys. Fonte: Própria.

Observa-se no recorte selecionado a presença do barramento principal do processador, os multiplexadores de seleção de periféricos e parte da lógica que conecta o núcleo *TinyQV* às memórias ROM e RAM simuladas pelo módulo *sim_qspi.v*. Esses blocos confirmam que a síntese lógica foi concluída corretamente pelo Yosys.

A figura 8 representa uma parte do diagrama de síntese física gerado pelo nextpnr-ecp5 e Yosys. Novamente, a figura completa ultrapassa das dimensões deste documento, e um vídeo demonstrando sua complexidade está disponível em: https://github.com/GuilhermeSallesBlanco/ProjetoTinyQV/blob/main/VideoPhysical.mp4.

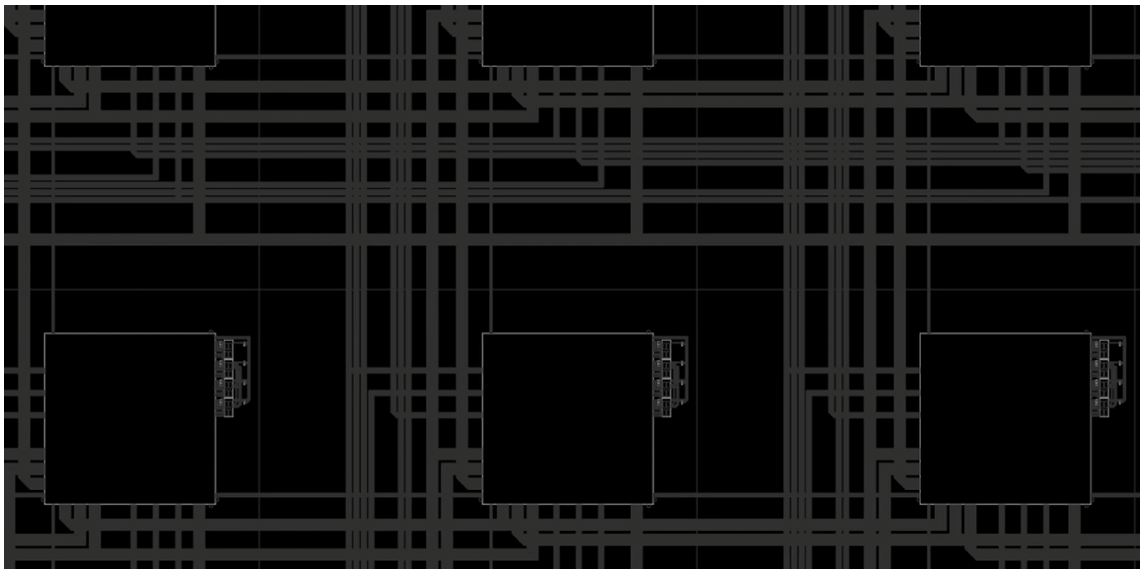

**Figura 8:** Diagrama de síntese física gerado pelo nextpnr-ecp5. Fonte: Própria.

O recorte evidencia regiões de lógica mapeada em LUTs, assim como trechos de interconexão roteados pelo nextpnr-ecp5. A disposição dos blocos indica que o núcleo foi corretamente posicionado e que a ferramenta conseguiu atender às restrições de pinos definidas no arquivo LPF.

A figura 9 demonstra a validação física final do sistema, evidenciando o LED conectado ao pino J16 da *Colorlight-i9*, que, dentro do arquivo LPF, representa a saída uo_out[1]. Um vídeo do funcionamento do LED está disponível em: https://github.com/GuilhermeSallesBlanco/ProjetoTinyQV/blob/main/videoLED.mp4.

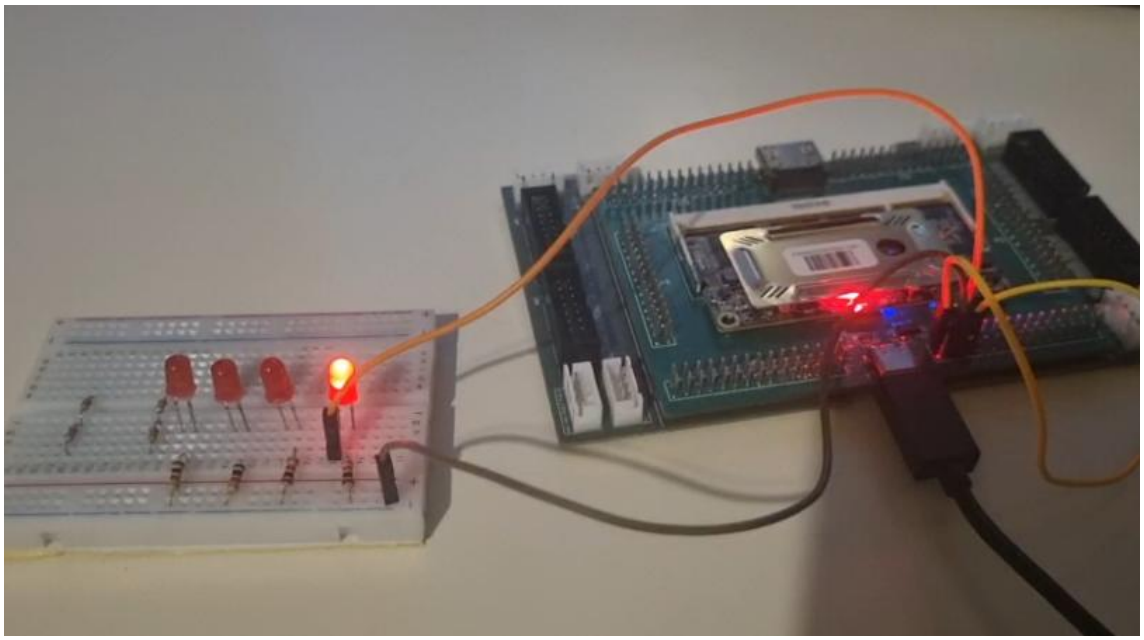

**Figura 9:** Funcionamento do programa de piscar LED. Fonte: Própria.

A frequência foi alterada dentro do programa C responsável por fazer o LED piscar, que piscou mais rapidamente ou mais lentamente, de acordo com as alterações feitas.

# 5) Conclusões

O objetivo deste trabalho foi, principalmente, portar, sintetizar e validar o funcionamento do processador RISC-V *TinyQV* em uma FPGA real, demonstrando a viabilidade prática de arquiteturas *open source* e comprovando a portabilidade do *TinyQV*. Apenas o processo de portação apresentou diversos problemas, como a necessidade de adaptar o processador e seus módulos para uma plataforma física específica, significando que não havia guias ou instruções exatas de como realizar esse processo. Além disso, foi necessário realizar a cadeia de síntese, roteamento, geração de *bitstream* e teste funcional diversas vezes, realizando alterações incrementais para que o processo estivesse funcionando corretamente para a placa *Colorlight-i9*.

Ao longo do desenvolvimento, os objetivos propostos foram alcançados. Os arquivos *Verilog* foram adaptados corretamente e foram sintetizados por meio do programa *Yosys* para a geração de um arquivo *bitstream* coerente e funcional. Logo em seguida, este arquivo foi corretamente carregado à FPGA, que executou todos os módulos sintetizados como esperado.

O processador sintetizado precisava, necessariamente, executar um programa no formato de arquivo .hex para demonstrar que estava ativo e funcionando. Logo, o objetivo de estabelecer um fluxo de compilação C -> .hex/.elf/.bin também foi atingido após a instalação e correção do *toolchain* apropriado. Os resultados apresentados mostram que os programas escritos em C foram corretamente compilados e executados pelo processador, validando a integração entre o *software* e o *hardware*.

Como forma de comprovar que a síntese dos códigos *Verilog* funcionou corretamente, foi produzido um *layout* físico que condizia com os módulos inseridos e as restrições de pinos definidas pelo arquivo LPF.

Finalmente, o fato de o teste prático do programa de piscar LED ter sido realizado com sucesso confirmou que o sistema realiza a operação completa esperada de um processador, como o funcionamento dos caminhos de *clock*, *reset*, memória, GPIO e *pipeline* de execução.

Apesar dos resultados positivos, o trabalho apresenta certas limitações naturais. A solução foi validada com um conjunto pequeno de periféricos, concentrando-se em comprovar o funcionamento do processador, ao invés de realizar testes com interfaces mais complexas, como UART, SPI, PWM, etc. Estas interfaces não foram integradas ou testadas fisicamente por conta das dificuldades mencionadas para apenas portar o *TinyQV*, o que resultou na falta de tempo hábil para programas e testes mais complexos.

Ainda assim, o trabalho conseguiu demonstrar a viabilidade prática de executar um núcleo RISC-V em uma FPGA real utilizando um fluxo totalmente *open source*. Como trabalhos futuros, destaca-se a possibilidade de ampliar o escopo de testes e o conjunto de periféricos utilizados e, finalmente, realizar um projeto prático e funcional com a utilização completa do processador RISC-V *TinyQV*. Este projeto cumpriu sua proposta e estabeleceu uma base sólida para o desenvolvimento de sistemas embarcados mais avançados e contribuindo para a disseminação de tecnologias abertas no contexto de *hardware* digital.

# 6) Repositório do projeto (GitHub)

O repositório do projeto se encontra no seguinte endereço *web*:

https://github.com/GuilhermeSallesBlanco/ProjetoTinyQV

# 7) Referências e Fontes

[1] TinyTapeout. Repositório da submissão de projetos para o *TinyQV*, disponível em: https://github.com/TinyTapeout/ttsky25a-tinyQV/tree/main, acessado em 19/11/2025.

[2] MichaelBell. Repositório do *core* do *TinyQV*, disponível em: https://github.com/MichaelBell/tinyQV, acessado em 19/11/2025.

[3] AVELAR, J. N.; GOMES, G. P. Guia de Utilização da Colorlight i9 com Ferramentas *OpenSource*, Instituto de Computação – UNICAMP, 2025.

[4] MichaelBell. Repositório do SDK (Source Development Kit) do *TinyQV*, disponível em: https://github.com/MichaelBell/tinyQV-sdk/tree/ttsky25a, acessado em 20/11/2025.

[5] MichaelBell. Repositório do *toolchain* de compiladores C e C++ para RISC-V, disponível em: https://github.com/MichaelBell/riscv-gnu-toolchain, acessado em 20/11/2025.

[6] FELDGOISE, J.; Artigo *online* "*RISC-V: What it is and Why it Matters?*", CENTER FOR SECURITY AND EMERGING TECHNOLOGY – CSET, 2024, disponível em: https://cset.georgetown.edu/article/risc-v-what-it-is-and-why-it-matters/, acessado em 22/11/2025.

[7] RISC-V International. Repositório do RISC-V, disponível em: https://github.com/riscv, acessado em 22/11/2025.

[8] Yosys Headquarters. Página de introdução e apresentação de projetos *Yosys*, disponível em: https://github.com/YosysHQ, acessado em 22/11/2025.

[9] KelvinChung2000. Repositório da ferramenta de posição e roteamento de códigos *Verilog* para FPGAs da família *Lattice* ECP5 nextpnr, disponível em: https://github.com/YosysHQ/nextpnr, acessado em 23/11/2025.

[10] trabucayre. Repositório da ferramenta de carregamento de *bitstreams* para FPGAs openFPGALoader, disponível em: https://github.com/trabucayre/openFPGALoader, acessado em 24/11/2025.

[11] Global Market Insights. *RISC-V Market Size & Share 2025 – 2034*, disponível em: https://www.gminsights.com/industry-analysis/risc-v-market, acessado em 02/02/2026.